

JavaScript

Frontend Development

Core Language Fundamentals

JavaScript's core building blocks — variables (let/const, avoiding the older var due to its confusing scoping rules), functions (including arrow functions, which don't rebind this and are common in modern code), and control flow (if/else, loops, switch) — are the foundation everything else builds on. Understanding truthy/falsy values (empty string, 0, null, undefined, and NaN are all falsy; almost everything else is truthy) explains a lot of JavaScript's sometimes-surprising conditional behavior, like why if (someArray.length) works as a non-empty check.

The DOM and Events

The DOM is a tree of node objects representing the page, and JavaScript manipulates it to create interactivity. Beyond basic selection and updates, understanding event propagation matters: events bubble upward from the target element through its ancestors by default, which is why a single click listener on a parent container can handle clicks on any of its children (event delegation) — often more efficient than attaching a separate listener to every individual child element, and it automatically works for children added later.

Asynchronous JavaScript

Most real frontend work — fetching data from an API, waiting on a timer, responding to user input over time — is asynchronous. Promises represent a value that will be available in the future (either resolved successfully or rejected with an error), and async/await is syntax that lets asynchronous code read like ordinary sequential code while still being non-blocking underneath. A function marked async always returns a Promise, and await pauses execution inside that function (not the whole program) until the awaited Promise settles — this distinction between 'this function pauses' and 'the browser tab freezes' is one of the most commonly misunderstood parts of JavaScript.

Working with APIs

The Fetch API is the standard way to make HTTP requests from the browser. A typical pattern: call fetch(url), await the response, check response.ok (fetch does not reject on HTTP error status codes like 404 or 500 — only on network failure — so this check is easy to forget), parse the body (commonly via response.json()), and handle errors with try/catch around the whole sequence. Getting comfortable reading the actual response — status code, headers, and body — rather than assuming success is what separates code that works in the demo from code that survives real network conditions.

ES6+ Features

Modern JavaScript adds syntax that makes common patterns shorter and less error-prone:

- Destructuring — pulling values out of objects/arrays directly into named variables (const { name, email } = user).
- Spread/rest syntax (...) — copying/merging arrays and objects, or collecting extra function arguments into an array.
- Template literals — string interpolation with backticks and `${}` instead of concatenation.
- Optional chaining (?.) and nullish coalescing (??) — safely accessing potentially-missing nested properties and providing fallback values without verbose manual checks.
- Modules (import/export) — splitting code across files with explicit, statically analyzable dependencies, replacing older global-script patterns.

Learn More

MDN Web Docs — JavaScript Guide

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>

javascript.info — The Modern JavaScript Tutorial

<https://javascript.info/>

freeCodeCamp YouTube Channel

<https://www.youtube.com/@freecodecamp>